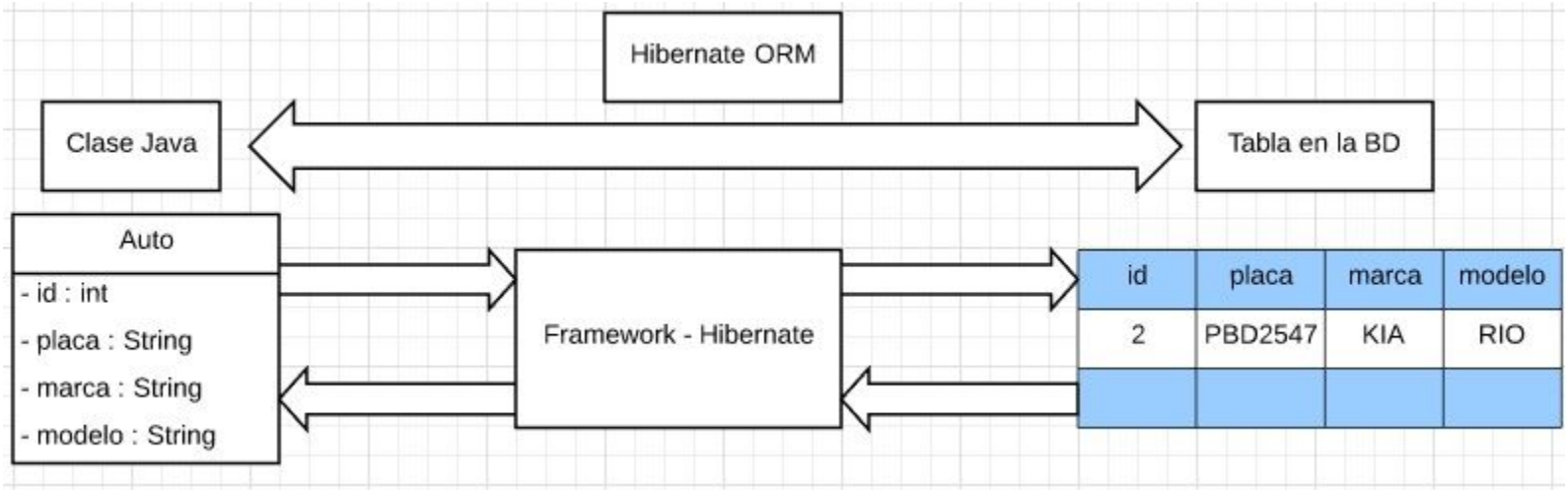
Spring data JPA relationships

¿Qué es JPA (Java Persistence API)?

- JPA es una especificación de una API que tiene como objetivo estandarizar la forma de acceder a una base de datos relacional desde Java mediante el ***mapeo relacional de objetos*** (ORM).
- A través de JPA, el desarrollador puede almacenar, eliminar, actualizar y recuperar datos de bases de datos relacionales a objetos Java y viceversa.
- La implementación de JPA se proporciona como una implementación de referencia por parte de los proveedores como **Hibernate**, **EclipseLink** y **Apache OpenJPA, DataNucleus**.

Mapeo Relacional de objetos (ORM)

Técnica utilizada para crear un mapeo entre una base de datos relacional y los objetos de un software, en nuestro caso, objetos Java y viceversa.

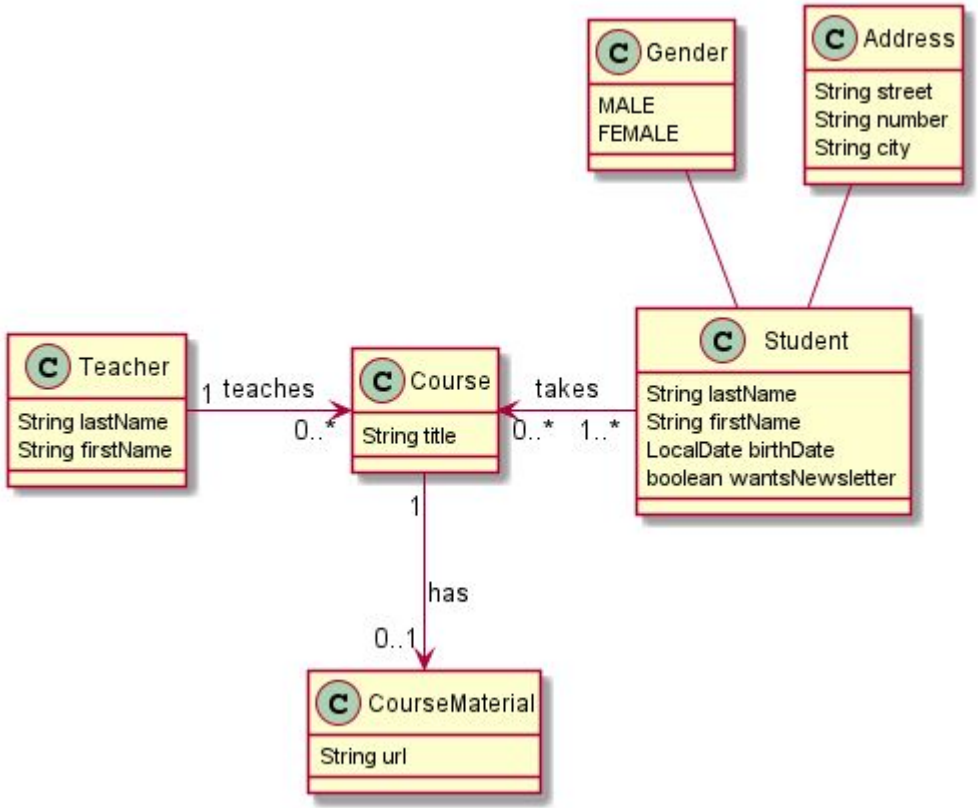


https://www.ecodeup.com/introduccion-a-jpa/

Relaciones entre entidades

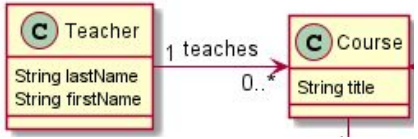
Tipos:

- One-to-Many
- Many-to-One
- One-to-One
- Many-to-Many



One-to-Many/Many-to-One

One-to-Many: es una relación que une a una entidad con otras muchas.




```java
@Getter
@Setter
@Entity
@NoArgsConstructor
@AllArgsConstructor
public class Teacher {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    @Column(nullable = false)
    private String firstName;

    @Column(nullable = false)
    private String lastName;

    @OneToMany(mappedBy = "teacher")
    private Set<Course> courses;

}
```

```java
@Getter
@Setter
@Entity
@Table(name = "courses")
@NoArgsConstructor
@AllArgsConstructor
public class Course {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    @Column(nullable = false)
    private String title;

    @JoinColumn(name = "teacher_id")
    private Teacher teacher;

}
```

Eager vs Lazy

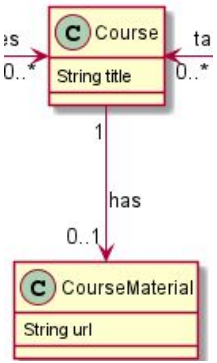
Lazy: significa que la relación no se cargará de inmediato, sino sólo cuando y si realmente se necesita.

Eager: significa que la relación se cargará de inmediato, es decir carga en memoria los datos desde la base de datos.

```java
public class Teacher {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    @Column(nullable = false)
    private String firstName;

    @Column(nullable = false)
    private String lastName;

    @OneToMany(mappedBy = "teacher", fetch = FetchType.EAGER)
    private Set<Course> courses;

```

```java
public class Course {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    @Column(nullable = false)
    private String title;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "teacher_id")
    private Teacher teacher;

}
```

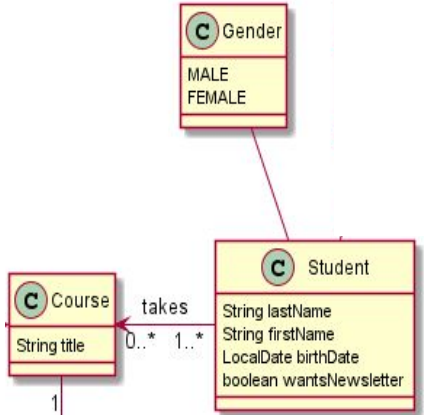
One-to-One




```java
public class Course {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    @Column(nullable = false)
    private String title;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "teacher_id")
    private Teacher teacher;

    @OneToOne(mappedBy = "course")
    private CourseMaterial material;

}
```

```java
@Data
@Entity
@Table(name = "course_material")
@NoArgsConstructor
public class CourseMaterial {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String url;

    @OneToOne(optional = false)
    @JoinColumn(name = "course_id", referencedColumnName = "id")
    private Course course;

}
```

Many-to-Many




```java
public enum Gender {
    MALE,
    FEMALE
}
```

```java
public class Course {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    @Column(nullable = false)
    private String title;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "teacher_id")
    private Teacher teacher;

    @OneToOne(mappedBy = "course")
    private CourseMaterial material;

    @ManyToMany
    @JoinTable(
      name = "STUDENTS_COURSES",
      joinColumns = @JoinColumn(name = "COURSE_ID", referencedColumnName = "id"),
      inverseJoinColumns = @JoinColumn(name = "STUDENT_ID", referencedColumnName = "id")
    )
    private List<Student> students = new ArrayList<>();

    public void addStudent(Student student){
        this.students.add(student);
    }
}
```

```java
@Builder
@Entity
@Table(name = "students")
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String lastName;

    @Column(name = "FIRST_NAME")
    private String firstName;

    @Column(name = "BIRTHDATE")
    private LocalDate birthDate;

    @Enumerated(EnumType.STRING)
    private Gender gender;

    private boolean wantsNewsletter;

    @ManyToMany(mappedBy = "students")
    private Set<Course> courses;

}
```

Implementación en Spring boot

Agregar el Driver de conexión correspondiente al motor de bd seleccionado en el archivo **pom.xml**

Para postgresql

```
<dependency>
<groupId>org.postgresql</groupId>
<artifactId>postgresql</artifactId>
<scope>runtime</scope>
</dependency>
```

Para H2 Database

```
<dependency>
<groupId>com.h2database</groupId>
<artifactId>h2</artifactId>
<scope>runtime</scope>
</dependency>
```

Implementación en Spring boot

Configurar la conexión a la bd en el archivo **application.properties**

Conexión para postgresql

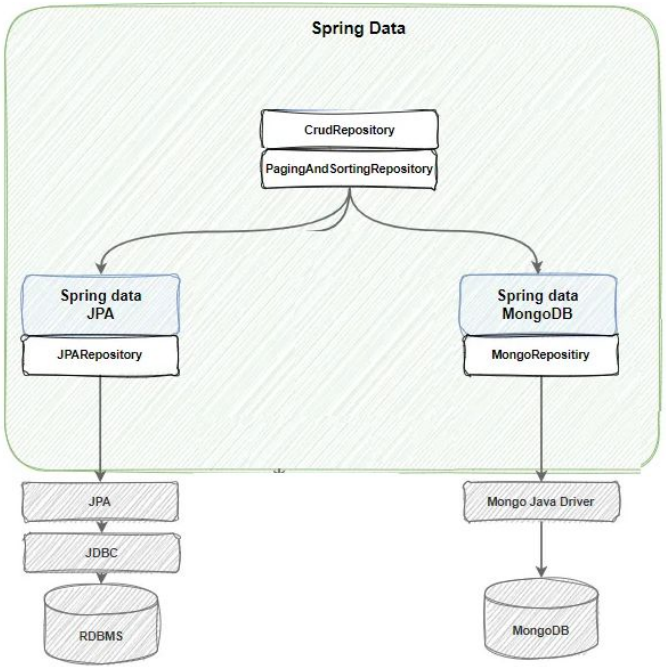
```
spring.datasource.url= jdbc:postgresql://localhost:5432/jpaexample
spring.datasource.username= postgres
spring.datasource.password= 123123
spring.jpa.properties.hibernate.jdbc.lob.non_contextual_creation= true
spring.jpa.properties.hibernate.dialect= org.hibernate.dialect.PostgreSQLDialect

spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Hibernate ddl auto (create, create-drop, validate, update)
spring.jpa.hibernate.ddl-auto= update
```

Conexión H2 Database

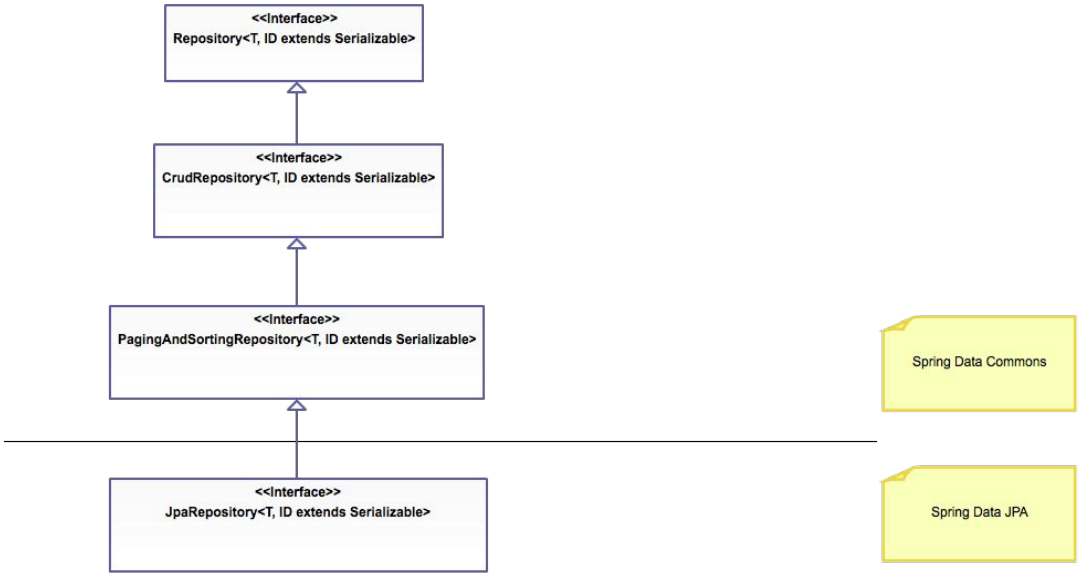
```
spring.datasource.url=jdbc:h2:file:/data/h2
spring.datasource.driverClassName:
org.h2.Driver
spring.datasource.username: sa
spring.datasource.password: password
spring.jpa.database-platform:
org.hibernate.dialect.H2Dialect
spring.h2.console.enabled: true
spring.jpa.show-sql=true
```

`jpaexample` es el nombre de la base de datos

Spring data Repository



adaptado de Spring Data Commons implementation for HAPI FHIR (Part 1) | by Svosh | Sep, 2023 | Medium



https://www.petrikainulainen.net/programming/spring-framework/spring-data-jpa-tutorial-introduction/

Referencias

Tutorial | Building REST services with Spring

https://stackabuse.com/a-guide-to-jpa-with-hibernate-relationship-mapping/

https://docs.spring.io/spring-data/jpa/docs/current/reference/html/

https://medium.com/huawei-developers/database-relationships-in-spring-data-jpa-8d7181f50f60

Getting Started with Spring Boot (reflectoring.io)